Jsp - Servlet

Chapitre 4: Suivi de la session

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW

1. Introduction

2. Les cookies
   - Création
   - Récupération

3. L'API Session
   - L'objet HttpSession
   - Etat de la session
   - Attributs de session

Introduction

- Le protocole HTTP est un protocole sans état, c'est à dire que le serveur ne peut pas faire le lien entre les requêtes successives d'un même client.
- Conséquence : le serveur ne peut pas suivre les actions du client.
- D'où la nécessité de créer des outils comme les cookies et l'API Session pour le suivi des actions du client.

Les cookies

- Un cookie est une petite information textuelle.
- Le cookie a un nom, une valeur et des propriétés.
- Il est crée au niveau du serveur et envoyé vers le client parmis les en-têtes HTTP de réponse.
- Le navigateur du client enregistre le cookie.
- A chaque requête vers le serveur, le navigateur ajoute le cookie parmis les en-têtes HTTP de requête.

Utilité du cookie

- Stocker des données chez le client.
- Suivre les activités du client.
- Se rappeler les préférences du client.
- Gérer la session.

Les cookies

Remarques :

- Le cookie peut être stocké chez le client plusieurs mois voire plusieurs années.
- Certains clients refusent les cookies parce qu'ils peuvent contenir des informations personnelles et être diffusés sur internet.

Création d'un cookie

- La création d'un cookie se fait avec le constructeur :

```
public Cookie(String nom, String valeur)
```

- L'envoie du cookie vers le client se fait avec la méthode (de `HttpServletResponse`) :

```
public void addCookie(Cookie cookie)
```

Exemple : Créer et envoyer un cookie vers le client

```
Cookie c = new Cookie("userId", "u235");
response.addCookie(c);
```

- Remarque :
  Si l'on oublie l'instruction `response.addCookie(c);`, le cookie ne sera pas envoyé vers le client.

Création d'un cookie

Remarques :

- Le cookie peut aussi être envoyé avec la méthode **addHeader** :

```
response.addHeader("Set-Cookie", "userId=u235");
```

  Mais il est préférable d'utiliser la méthode **addCookie**.

- Le cookie fait partie des en-têtes de réponse donc il doit être envoyé vers le client avant le corps de la réponse.

- Un navigateur doit pouvoir [1]
  - gérer des cookies d'au moins 4096 octets,
  - enregistrer au moins 20 cookies par serveur ou domaine
  - enregistrer au moins 300 cookies.

1. RFC 2965

Récupération d'un cookie

- Les cookies contenus dans la requête du client sont récupérés dans un tableau avec la méthode (**`HttpServletRequest`**) :

```
public Cookie[] getCookies()
```

Cette méthode retourne **`null`** si aucun cookie n'est contenu dans la requête.

- L'objet **`Cookie`** dispose des méthodes

```
public String getName()
```

et

```
public String getValue()
```

permettant respectivement de récupérer le nom et la valeur du cookie.

Récupération d'un cookie

Exemple : Récupérer le cookie nommé **"userId"** issu du client.

```
Cookie[] cookies = request.getCookies();
if(cookies != null){
    for(Cookie c : cookies){
        if(c.getName().equals("userId")){
            out.println(c.getName() + ": " + c.getValue() + "<br/>");
        }
    }
}
```

Caractérisques du Cookie

- Des caractérisques du cookie peuvent être définies avant son envoie vers le client :
  - domaine
  - chemin
  - protocole
  - durée de vie
  - sécurité

Caractérisques du Cookie

- Définir le domaine

```
public void setDomain(String nom_domaine)
```

  - Permet de définir la portée du cookie. Le rendre disponible pour tous les serveurs du domaine spécifié.
  - Le nom du domaine doit toujours commencer par un point.
  - La valeur par défaut est le nom du serveur ayant émis le cookie.

Exemple :

```
cookie.setDomain(.edu.sn);
```

indique que le cookie sera envoyé à tous les serveur du sous domaine `.edu.sn`.

Caractérisques du Cookie

- Définir le chemin

```
public void setPath(String uri)
```

  - Permet de définir la portée du cookie. Indique au navigateur d'ajouter le cookie aux requêtes contenant l'URI[2] spécifiée.
  - l'URI doit commencer par un slash (/)
  - La valeur par défaut est l'URI du servlet ayant émis le cookie.

Exemple :

```
cookie.setPath(/scolarite);
```

le cookie sera envoyée lorsque la requête contient l'URI `/scolarite`.

2. Uniform Resource Identifier

Caractérisques du Cookie

- Définir le protocole

```
public void setHttpOnly(boolean isHttpOnly)
```

  - Indique si le cookie ne doit être accessible que par voie HTTP(S).
  - Protéger le cookie contre un accès XSS (Cross-Site Scripting).

- Sécurisé ou non le cookie

```
public void setSecure(boolean secure)
```

  - Indique si le cookie ne doit être transmis que lorsque la connexion est sécurisée (https).
  - Si le contenu du cookie doit être crypté avant d'être envoyé vers le serveur.

Caractérisques du Cookie

- Définir la durée de vie

```
public void setMaxAge(int age_max)
```

  - Définit la durée de vie du cookie en seconde (cookie persistant).
  - Une valeur négative indique au navigateur d'effacer le cookie à sa fermeture (cookie session).
  - Une valeur nulle indique au navigateur d'effacer le cookie.

L'API Session

- L'API Session est une partie de l'API servlet dédiée au suivi de la session.
- A travers l'interface **`HttpSession`**, le serveur crée pour chaque client un objet session unique permettant de suivre ses activités.
- Avec cette API la gestion de la session repose sur les cookies ou sur la réécriture d'url.
- Par défaut le serveur utilise un cookie nommé **`JSESSIONID`** pour garder l'identifiant de la session.

L'API Session

- Lorsque le client n'accepte pas les cookies, le serveur bascule sur la réécriture d'url ; l'identifiant de la session est ajouté à l'url.
- Pour un bon suivi de la session, toutes les urls de l'application doivent être encodée avec la méthode (de **`HttpServletResponse`**)

```
public String encodeURL(String url)
```

Cette méthode ajoute si nécessaire l'identifiant de la session à l'url.

Exemple :

```
String urlEncode = response.encodeURL("scolarite");
out.println("<a href='"+ urlEncode +"'>scolarité</a>");
```

L'objet HttpSession

- L'objet session (**HttpSession**) associé au client est accessible à travers la requête émise par ce dernier.
- Le servlet récupère cet objet session grâce à la méthode (de **HttpServletRequest**)

```
public HttpSession getSession()
```

- Si la session de l'utilisateur n'est pas valide, la méthode **getSession** crée une nouvelle.
- Chaque objet session a un identifiant unique qui peut être connu avec la méthode (de **HttpSession**)

```
public String getId()
```

Etat de la session

Validité de la session :

- `public void setMaxInactiveInterval(int intervalle)`

  Permet de définir la durée d'inactivité de la session.

- `public int getMaxInactiveInterval()`

  Permet de connaitre la durée d'inactivité attribuée à la session.

  Remarque Lorsque la durée indiquée est nulle ou négative cela veut dire que la session n'expire jamais.

- `public void invalidate()`

  permet d'invalider une session. Elle est généralement utilisée pour déconnecter un utilisateur.

Etat de la session

L'état de la session peut être connu à l'aide des méthodes :

- `public boolean isNew()`

  permet de savoir si la session est nouvelle ou pas. Une session est nouvelle si elle n'est pas encore arrivée sur le client (le navigateur).

- `public long getCreationTime()`

  retourne la date de création de la session comme un `long` représentant le nombre de millisecondes écoulés depuis 1 jan. 1970.

- `public long getLastAccessedTime()`

  donne la date de dernier accès à la session en nombre de millisecondes écoulés depuis 1 jan. 1970.

Attributs de session

- Des objets peuvent être attachés à la session

```
public void setAttribute(String nom, Object valeur)
```

Exemple :

```
HttpSession session = request.getSession();
session.setAttribute("userId", "u235");
```

Remarque :

Si un objet du même nom existe déjà dans la session, il est remplacé par le nouveau.

Attributs de session

- Récupérer un objet de la session :

```
public Object getAttribute(String nom)
```

Exemple :

```
HttpSession session = request.getSession();
String userId = (String) session.getAttribute("userId");
```

Remarque :

Si l'objet n'existe pas dans la session, la méthode **`getAttribute`** retourne **`null`**.

Attributs de session

- L'ensemble des noms des objets stockés dans la session peut être obtenu avec la méthode :

```
public Enumeration<String> getAttributeNames()
```

- La suppression d'un objet de la session est réalisée avec la méthode :

```
public void removeAttribute(String nom)
```

- Remarques
  - La méthode `removeAttribute` lance une exception lorsqu'elle est appelée sur une session invalide.
  - Lorsque la session expire ou est invalidée les objets qui lui étaient associés disparaissent avec elle.